Shakti E-Class (E32-A35 / SP2020) Simulation Guide: Steps 1 to 8

A simple, direct step-by-step document to build, accelerate, and simulate the pure Shakti E-Class Pinaka SoC with Icarus Verilog (`iverilog` + `vvp`).

Step 1: Clone SP2020 & Navigate to E32-A35

Clone the official IIT Madras repository and enter the Pinaka SoC directory:

```
bash
git clone https://gitlab.com/shaktiproject/sp2020.git
cd sp2020/e32-a35
```

Step 2: Apply the 3 Compatibility Fixes

Apply non-invasive fixes for open-source BSC and multilib GCC toolchains:

```
bash
# 1. Fix BSC heap limit (-K4000M) and remove obsolete basicinout filter
sed -i 's/-K40000M/-K4000M/g' Makefile
sed -i 's|-verilog-filter [^ ]*||g' Makefile

# 2. Fix GCC ABI in Boot Code (-mabi=ilp32 and remove picolibc.specs)
sed -i 's/--specs=picolibc.specs//g' boot-code/Makefile
sed -i 's/-march=rv$(XLEN)imac/-march=rv$(XLEN)imac -mabi=ilp32/g' boot-code/Makefile

# 3. Add pipelined multiplier model (if not already present)
# File: common_verilog/multiplier.v
```

Step 3: Generate Verilog from Bluespec BSV

Compile the high-level BSV hardware description into cycle-accurate Verilog modules:

```
bash
make generate_verilog CONFIG=core_config.inc
```

Outputs generated in `verilog/*.v`: `mkSoc.v`, `mkeclass.v`, `mkmuldiv.v`, `mkuart.v`, `mkicache.v`, `mkdcache.v`.

Step 4: The 81.5x Acceleration Trick (32-Clock Baud System)

In hardware at 19,200 baud, 1 bit = 2,608 clock cycles (26,080 cycles/character). Set the software baud divisor to `2` to make it 81.5x faster (32 cycles/bit = 320 cycles/character):

```
bash
# Set baud divisor to 2 in boot code
for bfile in boot-code/*.c boot-code/*.h; do
    sed -i 's/\b163\b/2/g' "$bfile" || true
    sed -i 's/\*uart0_baud *= *[0-9]*;/\*uart0_baud = 2;/g' "$bfile" || true
done

# Compile Boot ROM code into boot.mem
make generate_boot_files CONFIG=core_config.inc
```

Step 5: Prepare Simulation Workspace

Create an isolated simulation directory and preload memory files:

```
bash
mkdir -p sim_baud_fix
cp boot-code/outputs/boot.mem sim_baud_fix/boot.mem
touch sim_baud_fix/code.mem sim_baud_fix/code1.mem
```

Step 6: Compile Complete SoC with Icarus Verilog

Elaborate all Verilog modules and build the executable simulation model `shakti_soc.vvp`:

```
bash
BSV_VDIR="/usr/local/lib/Verilog"
[ ! -d "$BSV_VDIR" ] && BSV_VDIR="/usr/local/lib/Libraries/Verilog"

iverilog -g2012 \
    -o sim_baud_fix/shakti_soc.vvp \
    -I ${BSV_VDIR} -y ${BSV_VDIR} \
    -I verilog -y verilog \
    -I common_verilog -y common_verilog \
    -I devices -y devices -I fabrics -y fabrics \
    /workspaces/shakti-riscv-codespace/baud_fix/multiplier.v \
    /workspaces/shakti-riscv-codespace/baud_fix/uart_rx_monitor.v \
    /workspaces/shakti-riscv-codespace/baud_fix/tb_soc_baud_fix.v \
    verilog/*.v
```

Step 7: Run Cycle-Accurate Simulation with `vvp`

Run the simulation. Because the baud is set to 32 clocks/bit, the boot sequence executes in **3 to 5 seconds**:

```
bash
cd sim_baud_fix
vvp shakti_soc.vvp
```

Step 8: Expected Console Output

```
text
[TB] System reset released. Starting Fast-Baud Shakti SP2020 simulation...

                        .///*
                     ,(((((,
                   *((((((,
               ./(((((((,
             ./(((((((((*
          *((((((((((/*
       .((((((((((,
    ,((((((((((((/
  ((((((((((((((/
 .((((((((((((((/
    *((((((((((.
        /(((((((.
         ..   *(((
             *((,   ,/
               ((((((/.
               ((((((((((/
                ((((((((((((/
                ((((((((((((.
               ((((((((((((/
             *((((((((((*
             ((((((((((.
          /(((((((((*
       *(((((((,
     *((((((.
  .(((((
./(((*
.

SHAKTI PROCESSORS

Booting... vspil.0

Booting on Pinaka! hart 0
Pinaka is a SoC build on top of Artix7 35T.
The core belongs to Shakti E class, 32 bit.

Supported ISA: RV32IMACNU.
Processor Arch ID: 0.
Shakti booted

   6733200 SPI : Transmit state is in idle
```

```
 id ffff
No content to boot
```

Step 9: Compile Pure Base ISA Core (RV32I Minimal Configuration)

To strip the processor down to a minimal **Base RV32I core** (removing M, A, C extensions and cutting core area by ~60%):

1. What Lines are Changed in `core_config.inc`:

Line / Parameter	Original SP2020 (`RV32IMAC`)	Changed Base Core (`RV32I`)	Impact
`ISA`	`RV32`	`RV32I`	Strips M, A, C instruction decoders
`MUL`	`enable`	`disable`	Completely eliminates hardware multiplier
`DIV`	`enable`	`disable`	Completely eliminates hardware divider
`ATOMIC`	`enable`	`disable`	Removes LR/SC and AMO memory logic
`COMPRESSED`	`enable`	`disable`	Standard 32-bit instructions only
`USER`	`enable`	`disable`	Machine-mode (M-mode) only
`MULSTAGES`	`4`	`0`	Multiplier latency (0 = disabled, 1/2/4 for stages)
`DIVSTAGES`	`32`	`0`	Multi-cycle divider stages

2. Can We Add Stages Here?

- **Execution Unit Stages (`MULSTAGES` / `DIVSTAGES`): YES.** You can configure the latency/pipelining of functional units:
  - `MULSTAGES=0`: Disabled (Base RV32I).
  - `MULSTAGES=1`: Single-cycle combinatorial multiplier.
  - `MULSTAGES=2` or `MULSTAGES=4`: 2 or 4-stage pipelined multiplier (higher clock speed). When set to `4`, your testbench/simulation must use a 4-stage pipelined multiplier model (`multiplier.v`).

- **Core Pipeline Stages (Fetch → Execute → Writeback):** Shakti E-Class has an in-order **3-stage microarchitecture** written in Bluespec SystemVerilog (BSV). You cannot convert the 3-stage core into a 5-stage core via `core_config.inc` alone because the pipeline stages are hard-coded in the BSV stage rules. (For 5-stage or out-of-order 6-stage designs, Shakti provides the **C-Class** and **I-Class** cores).

3. Pure Base ISA `core_config.inc`:

```
makefile
ISA=RV32I
MUL=disable
DIV=disable
ATOMIC=disable
COMPRESSED=disable
USER=disable
MULSTAGES=0
DIVSTAGES=0
```

2. **Regenerate Verilog from Bluespec:**

```
bash
make clean
make generate_verilog CONFIG=core_config.inc
```

(Notice: `mkmuldiv.v` is no longer generated! Zero DSP slices or multiplier models needed).

3. **Compile Software with Base ISA Flags:**

```
bash
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -O2 boot.c -o boot.elf
```